

Laboratory 06

WCET Measurement with DWT & Interrupt Latency

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXXN236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Estimated time	3–4 hours
CLO mapping	CLO 3 & CLO 4

1. Objectives

By completing this lab you will be able to:

1. **Enable and read** the DWT cycle counter on the ARM Cortex-M33 at 150 MHz.
2. **Measure** best-case, average, and worst-case execution time for a data-processing function over 1 000 runs.
3. **Apply** a WCET safety margin and use the result as C_i in schedulability analysis.
4. **Measure** interrupt latency from IRQ assertion to first ISR instruction using an oscilloscope or logic analyser.
5. **Compare** measured interrupt latency with and without a FreeRTOS critical section active.

2. Prerequisites

- ☐ Lab 03 complete.
- ☐ ARM GNU Toolchain and pyocd working.
- ☐ Oscilloscope or logic analyser with ≥ 4 channels (for Part B). A probe on GPIO pins is sufficient.
- ☐ FRDM-MCXXN236 board — no other hardware required for Part A.

3. Background

3.1. Why Measure WCET?

The Response-Time Analysis from Week 3 requires a bound C_i on task execution time. In practice, C_i is obtained by:

1. **Static analysis** — examine all paths, compute worst-case bound. Requires special tools (aiT, Bound-T).
2. **Measurement-based** — run the function many times with controlled inputs, take the maximum plus a safety margin.

This lab uses measurement-based WCET — practical for most systems below SIL-3/ASIL-D.

3.2. DWT Cycle Counter

The ARM Data Watchpoint and Trace (DWT) unit has a 32-bit free-running cycle counter. Steps to enable:

```
1 CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;  /* enable DWT */
2 DWT->CYCCNT = 0;
3 DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;             /* start counting
   */
```

Read: `uint32_t cycles = DWT->CYCCNT;`

At 150 MHz, 1 cycle = 6.67 ns. The counter rolls over after 2^{32} cycles $\approx$ 28.6 seconds.

4. Project Structure

```
lab06_wcet_dwt/
  Makefile
  linker_MCXN236.ld
  src/
    main.c           <- DWT init, task creation
    dwt.h            <- DWT enable/read macros
    workload.c/.h    <- the functions to be measured
    latency_test.c/.h <- Part B: GPIO ISR latency test
    FreeRTOSConfig.h
    uart.h / uart.c
    FreeRTOS-Kernel/
```

5. Part A — WCET Measurement

5.1. Step 1 — Enable DWT

In `dwt.h`:

```
1 #ifndef DWT_H
2 #define DWT_H
3 #include "core_cm33.h"
4
5 static inline void DWT_Enable(void)
6 {
7     CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
8     DWT->CYCCNT = 0;
9     DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
10    __DSB();
11    __ISB();
12 }
13
14 static inline uint32_t DWT_GetCycles(void)
15 {
16     return DWT->CYCCNT;
17 }
18
19 #define DWT_START()    uint32_t _dwt_start = DWT_GetCycles()
20 #define DWT_STOP()     uint32_t _dwt_elapsed = DWT_GetCycles() -
21                        _dwt_start
22 #endif
```

5.2. Step 2 — Implement the Workload Functions

Three functions of increasing complexity in `workload.c`:

```
1 /* Function 1: bubble sort -- data-dependent WCET */
2 void bubble_sort(int16_t *arr, uint16_t n)
3 {
4     for (uint16_t i = 0; i < n - 1; i++)
5         for (uint16_t j = 0; j < n - i - 1; j++)
6             if (arr[j] > arr[j+1]) {
7                 int16_t tmp = arr[j];
8                 arr[j] = arr[j+1];
9                 arr[j+1] = tmp;
10            }
11 }
12
13 /* Function 2: 16-tap FIR filter -- fixed WCET */
```

```

14 int32_t fir_filter(const int16_t *x, const int16_t *h, uint16_t n)
15 {
16     int32_t acc = 0;
17     for (uint16_t i = 0; i < n; i++)
18         acc += (int32_t)x[i] * h[i];
19     return acc >> 15;
20 }
21
22 /* Function 3: 32-bit CRC computation -- data-dependent */
23 uint32_t crc32(const uint8_t *data, uint32_t len)
24 {
25     uint32_t crc = 0xFFFFFFFFU;
26     while (len--) {
27         crc ^= *data++;
28         for (int k = 0; k < 8; k++)
29             crc = (crc >> 1) ^ (0xEDB88320U & ~(crc & 1));
30     }
31     return ~crc;
32 }

```

5.3. Step 3 — Measurement Task

```

1  #define NUM_RUNS      1000
2  #define ARRAY_SIZE    100
3
4  void vWcetMeasureTask(void *pv)
5  {
6      static int16_t sorted[ARRAY_SIZE], reverse[ARRAY_SIZE], arr[
7          ARRAY_SIZE];
8      static int16_t fir_h[16] = {1,2,3,4,5,6,7,8,8,7,6,5,4,3,2,1};
9      uint32_t best, worst, sum, cycles;
10
11     for (int i = 0; i < ARRAY_SIZE; i++) {
12         sorted[i] = (int16_t)i;
13         reverse[i] = (int16_t)(ARRAY_SIZE - 1 - i);
14     }
15
16     uart_printf("\r\n=== bubble_sort(%d elements) ===\r\n",
17         ARRAY_SIZE);
18
19     /* Worst case: reverse sorted, NUM_RUNS times */
20     best = UINT32_MAX; worst = 0; sum = 0;
21     for (int r = 0; r < NUM_RUNS; r++) {
22         memcpy(arr, reverse, sizeof(reverse));
23         DWT_START();
24         bubble_sort(arr, ARRAY_SIZE);
25         DWT_STOP();
26         cycles = _dwt_elapsed();
27         if (cycles < best) best = cycles;
28         if (cycles > worst) worst = cycles;
29         sum += cycles;
30     }
31     uart_printf("  best=%lu  worst=%lu  avg=%lu  cycles\r\n",
32         best, worst, sum / NUM_RUNS);
33     uart_printf("  WCET with 20%% margin: %lu cycles  (0.2f us)\r\n",
34         (uint32_t)(worst * 1.2f), worst * 1.2f / 150.0f);

```

```

34     uart_printf("\r\nMeasurement complete.\r\n");
35     vTaskDelete(NULL);
36 }

```

5.4. Step 4 — Build, Flash, and Record Results

```
make setup && make && make flash
```

✓ Checkpoint A

Terminal output with all three function measurements.

? Question A1

Bubble sort has the same code path for all reverse-sorted inputs. Why does **worst** vary across the 1 000 runs even with identical input? Name two hardware effects that cause this variability.

? Question A2

The FIR filter is data-independent (no branches). Does it show variability across runs? What is the ratio of worst to best cycles? Explain.

? Question A3

You will use `bubble_sort` as a task in Lab 07 with period $T = 50$ ms. Using the WCET + 20% margin as C_i : compute the utilisation C_i/T_i . At what maximum number of identical `bubble_sort` tasks does the total utilisation exceed the Liu & Layland bound for RMS?

6. Part B — Interrupt Latency Measurement

6.1. Step 1 — Set Up the GPIO Toggling ISR

Wire a jumper from P1_0 (output) to P1_1 (input, interrupt on rising edge). The ISR immediately toggles P1_2 (output to oscilloscope probe).

```

1  /* latency_test.c */
2  void GPIO1_IRQHandler(void)
3  {
4      GPIO_ClearPinsInterruptFlags(GPIO1, 1u << 1);
5      GPIO_TogglePinsOutput(GPIO1, 1u << 2);    /* respond ASAP */
6  }

```

6.2. Step 2 — Configure GPIO and NVIC

```

1  /* Input: P1_1 rising edge interrupt */
2  gpio_pin_config_t in_cfg = { kGPIO_DigitalInput, 0,
3                               kGPIO_IntRisingEdge };
4  GPIO_PinInit(GPIO1, 1, &in_cfg);
5  GPIO_EnableInterrupts(GPIO1, 1u << 1);
6  EnableIRQ(GPIO1_IRQn);
7
8  /* Output: P1_2 */
9  gpio_pin_config_t out_cfg = { kGPIO_DigitalOutput, 0,
10                               kGPIO_NoIntmode };
11 GPIO_PinInit(GPIO1, 2, &out_cfg);

```

6.3. Step 3 — Drive the Input and Measure

Three measurement options:

- **Option A — Logic analyser:** toggle P1_0 at 1 kHz from a software task; measure P1_1-rising to P1_2-rising.
- **Option B — Oscilloscope:** probe P1_1 (CH1) and P1_2 (CH2); trigger on CH1 rising; measure propagation delay.
- **Option C — DWT inside the ISR:** record DWT->CYCCNT as the first ISR instruction and compare against the cycle count set just before toggling P1_0.

6.4. Step 4 — Measure With and Without Critical Section

Repeat the measurement with a FreeRTOS critical section active in a background task:

```

1 void vCriticalSectionTask(void *pv)
2 {
3     for (;;) {
4         taskENTER_CRITICAL();
5         volatile uint32_t i = 0;
6         while (i++ < 150000) {} /* ~1 ms at 150 MHz */
7         taskEXIT_CRITICAL();
8         vTaskDelay(pdMS_TO_TICKS(5));
9     }
10 }
```

✓ Checkpoint B

Table or oscilloscope screenshot showing latency with and without the critical section.

? Question B1

Your measured best-case latency should be 12–18 cycles. Convert this to nanoseconds at 150 MHz. How does this compare to the theoretical minimum from the ARM Cortex-M33 TRM (exception entry stacking)?

? Question B2

With the critical section active, what is the worst-case interrupt latency? Where does the extra latency come from? What FreeRTOS configuration parameter controls this?

? Question B3

The ISR priority is set to `configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY - 1`. Why can this ISR still be deferred by `taskENTER_CRITICAL`? What priority would you set to make the ISR **non-maskable** by FreeRTOS?

7. Part C — Full Task WCET in Context

7.1. Step 1 — Measure Task Response Time End-to-End

Create a periodic task that runs `bubble_sort` on random data every 50 ms. Use DWT to measure from task entry to task return, with two background tasks also active (priorities 1 and 2):

```

1 void vSortTask(void *pv)
2 {
3     static int16_t arr[ARRAY_SIZE];
4     TickType_t xLastWake = xTaskGetTickCount();
5     for (;;) {
6         vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(50));
```

```

7      for (int i = 0; i < ARRAY_SIZE; i++) arr[i] = (int16_t)(rand
      () % 1000);
8      DWT_START();
9      bubble_sort(arr, ARRAY_SIZE);
10     DWT_STOP();
11     uart_printf("[SORT] %lu cycles  (%.2f us)\r\n",
12                 _dwt_elapsed, _dwt_elapsed / 150.0f);
13 }
14 }

```

Compare the measured cycles to Part A's isolated measurement.

? Question C1

Does running alongside other tasks increase the measured execution time? What hardware effect causes this? What does this tell you about measurement-based WCET methodology?

8. Deliverables

#	Item	Details
1	Part A terminal output	Full measurement table for all three functions
2	Part A analysis	WCET + 20% margin and schedulability (Question A3)
3	Part B measurement	Latency table or oscilloscope screenshot
4	Part B analysis	With and without critical section (Question B2)
5	Written answers	Questions A1–A3, B1–B3, C1
6	dwt.h header file	Full source listing

9. Key Concepts Demonstrated

Concept	Where you saw it
DWT cycle counter enable and read	Part A setup
Best/average/worst-case measurement	Part A
WCET safety margin application	Part A Question A3
Interrupt latency: hardware stacking	Part B
BASEPRI masking and FreeRTOS critical section	Part B
Context and cache effects on WCET	Part C

10. Reference

Resource	Relevant sections
Wilhelm et al. (2008)	“The worst-case execution time problem” (<i>ACM TECS</i>)
ARM Cortex-M33 TRM	§DWT, §B2.3 exception entry
NXP MCXN236 RM	§CoreDebug, §DWT
FreeRTOS docs	<code>taskENTER_CRITICAL</code> , <code>configMAX_SYSCALL_INTERRUPT_PRIORITY</code>
